科目：データベースの基礎

# 第4回 リレーショナルデータモデル：操作記述

講師：鈴木源吾

# 前回の復習

1. リレーショナルデータモデルの制約
    - 候補キー・主キー・キー制約．一意に識別できるかは属性の意味で決まる
    - 外部キー・外部キー制約（参照整合性制約）．空値（NULL）・NOT NULL制約
2. その他の一貫性制約とSQLによる制約記述
    - ドメイン制約・UNIQUE制約・検査制約（CHECK）．CREATE TABLE の中に書く

# 第3回演習の解説

- 第3回ポイント資料の裏面（演習1〜3・確認問題）の正解を解説する
- 解説は，演習の締切（講義開始30分後）のあとに行う
- 解答例は電子的には配布しない．**自分の解答に転記すること**（解説中の撮影は不可）
  - 講義中に自分の解答へ書き足す・訂正するのはよい．そのまま試験に持ち込める
- 提出がまだの人は，WebClass の「【締切後】第3回 演習」から提出する（0.7点で評価）

# 今回のトピック

0. 実習環境の確認（冒頭5分）
1. リレーショナルデータモデルの操作：リレーショナル代数
2. SQLについて：背景・歴史・利用法など
3. SQLの基本：SELECT文による検索
4. 今回の演習と小テストの「型」

教科書：第4章 p.34〜47

# 実習環境の確認（5分）

今回から **Docker の PostgreSQL（OS入門で構築）** を使う．手順は学生ポータルの「SQL 実習環境ガイド」

1. Docker Desktop を起動 → ターミナルで `docker start pg18`
2. pgAdmin を起動 → `pg18` に接続 → `postgres` を選んで **Tools → Query Tool**
3. 次を入力して実行（F5）．`PostgreSQL 18.x ...` と出れば OK

```
SELECT version();
```

- **できない人は挙手** → 講義中に SA が個別に対応する（SA がいるのは今回だけ）
- 解決するまでは **Colab**（ポータル第4回のノートブック．SQLite）で進める．確認は `SELECT sqlite_version();`（`3.xx.x` と出れば OK）．ほかの SQL は同じ

Topic 1

# リレーショナルデータモデルの操作

## トピックのキーワード

- リレーショナル代数
- 集合演算（和・差・共通・直積）と和両立
- 射影・選択・結合（等結合・自然結合）・商

# リレーショナル代数（relational algebra）

- リレーショナルデータモデルを操作する「**基礎となる**」演算の体系
- DBMSの利用者が直接使うわけではない
- SQL言語は，リレーショナル代数の考え方を基本にして設計されている
- リレーションに演算を施すと結果もリレーション（**再帰性**と呼ぶ）
- 再帰性はRDBMSの様々な機能に役立っている
  - 例：ビュー：仮想的なリレーションを定義する仕組み
  - 例：最適化：演算を交換して高速に処理

# リレーショナル代数の演算

- 集合演算
  - 和集合演算
  - 差集合演算
  - 共通集合演算
  - 直積演算
- リレーショナル代数に特有の演算
  - 射影演算
  - 選択演算
  - 結合演算
  - 商演算

# 和集合演算 (Union)

- 「同じリレーションスキーマである」リレーションのタプルを合わせ一つのリレーションにする
  - 表記法：$R \cup S$
  - 厳密には，同じリレーションスキーマである必要はなく，2つのリレーションは「**和両立**」であれば良い（次数とドメインが等しい．詳細は教科書参照）

R

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |

＋

S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |



R ∪ S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |

# 差集合演算 (Set Difference)

- あるリレーションから，もう一つのリレーションを取り除く
  - 表記法：$R - S$
  - 和集合演算と同様に2つのリレーションが和両立である必要がある

R

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |

—

S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |



R — S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |

# 差集合演算の注意

- Sのタプルのうち，Rにはないタプルは「引けない」
- つまり，R−Sのタプルは，Rのタプルであって，Sのタプルではないもの

R

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |

−

S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |
| 00005 | 石原 | 良純 | 環境情報 | 情報 | 2009 |

R − S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |

# 共通集合演算 (Intersection)

- 2つのリレーションの共通部分
  - 表記法：$R \cap S$
- 和集合演算と同様に2つのリレーションが和両立である必要がある

R

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |

∩

S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |
| 00005 | 石原 | 良純 | 環境情報 | 情報 | 2009 |

R ∩ S

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |

# 直積演算 (積，Cartesian Product)

- 2つのリレーションの全部のタプルの組み合わせをタプルにしたリレーションを作成する
  - 表記法：$R \times S$
  - タプル数は，Rのタプル数 × Sのタプル数

R

| A | B |
|---|---|
| 1 | 2 |
| 3 | 4 |

×

S

| B | C | D |
|---|---|---|
| 2 | 5 | 6 |
| 4 | 7 | 9 |
| 9 | 10 | 11 |



R × S

| A | R.B | S.B | C | D |
|---|---|---|---|---|
| 1 | 2 | 2 | 5 | 6 |
| 1 | 2 | 4 | 7 | 9 |
| 1 | 2 | 9 | 10 | 11 |
| 3 | 4 | 2 | 5 | 6 |
| 3 | 4 | 4 | 7 | 9 |
| 3 | 4 | 9 | 10 | 11 |

- 複数のテーブルを使った処理の基本
- しかし、結果のタプル数が大きくなる
- 実際には、直積を使わず、次の結合を使うことが多い

# 射影演算 (Projection)

- リレーションの特定の属性だけを抜き出す
  - 表記法：$\pi_{\text{属性名}}(R)$（属性名は複数指定できる）
  - 教科書の表記法だと R[属性名]．$\pi$（パイ）を使うほうが一般的
- **結果はリレーション（集合）なので，重複するタプルは1つにまとまる**

R

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |



$\pi_{学籍番号, 姓, 名}(R)$

| 学籍番号 | 姓 | 名 |
|---|---|---|
| 00001 | 秋吉 | 久美子 |
| 00002 | 赤坂 | 泰彦 |
| 00003 | 飯島 | 愛 |
| 00004 | 泉 | ピン子 |

# 選択演算 (Selection)

- リレーションから条件を満たすタプルを選ぶ
  - 表記法：$\sigma_{\text{条件式}}(R)$
  - 教科書の表記法だと R[条件式]
- 属性はすべて残る（行を選ぶだけ．列は減らない）

R

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 環境情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 環境情報 | 情報 | 2008 |
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |
| 00005 | 石原 | 良純 | 環境情報 | 情報 | 2009 |
| 00006 | 伊東 | 四朗 | 環境情報 | メディア | 2010 |
| 00007 | うつみ | 宮土理 | 環境情報 | 情報 | 2010 |

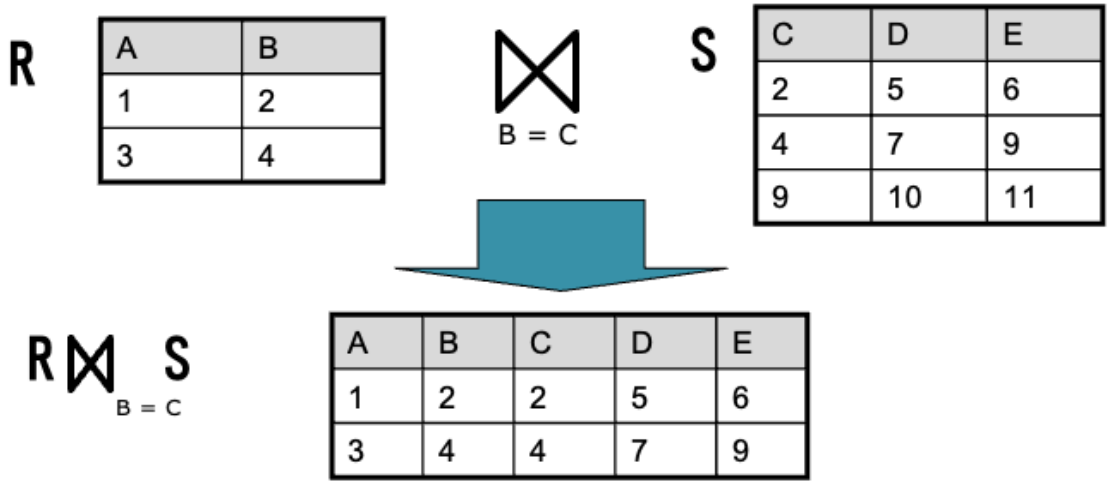
$\sigma_{\text{学籍番号} \geq 00003 \text{ AND } \text{学籍番号} < 00005}(R)$



| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00003 | 飯島 | 愛 | 環境情報 | メディア | 2009 |
| 00004 | 泉 | ピン子 | 環境情報 | メディア | 2009 |

# 結合演算 (Join)

- 2つのリレーションをある属性間の関係でお互いのタプルを結合し関連づける演算
- 表記法：$R \bowtie_{\text{条件式}} S$
- 条件式で比較する属性はドメインが等しい必要がある（例：数値と文字はNG）
- 最もよく用いられる条件式は，属性の値が等しい場合（**等結合**（equi join））
- 等号以外の一般的な条件式の結合をΘ結合（シータ結合）と呼ぶことがある



# 自然結合演算 (Natural Join)

- 等結合では，条件式に使った属性の両方が表示されるため，同じ値が並ぶ
  - 例）前ページのBとC
- 自然結合は，「名前が同じ」属性で結合する演算
- 結果的に，重複した属性は発生しない（同じ名前だから）
- 結合相手がないタプルは結果に残らない（空値で埋めたりしない）

R

| A | B |
|---|---|
| 1 | 2 |
| 3 | 4 |

$\bowtie$

S

| B | C | D |
|---|---|---|
| 2 | 5 | 6 |
| 4 | 7 | 9 |
| 9 | 10 | 11 |



$R \bowtie S$

| A | B | C | D |
|---|---|---|---|
| 1 | 2 | 5 | 6 |
| 3 | 4 | 7 | 9 |

# 商演算（Division）

- 定義は教科書参照（難しいので，以下の例で理解しよう）
  - 表記法：$R \div S$
- 例：「登録されているすべてのプロジェクトのメンバとなっている人の従業員番号を求める」
  - 割るリレーションで分類して全部に値ある→割り切れる，というイメージ
  - それほどは使われない

プロジェクトメンバ

| プロジェクト番号 | 従業員番号 |
|---|---|
| p1 | 1 |
| p1 | 2 |
| p1 | 3 |
| p2 | 2 |
| p2 | 3 |
| p3 | 2 |
| p3 | 3 |

プロジェクト

| プロジェクト番号 |
|---|
| p1 |
| p2 |
| p3 |

プロジェクトメンバ
÷プロジェクト

| 従業員番号 |
|---|
| 2 |
| 3 |

# リレーショナル代数の演算の複数適用

**リレーショナル代数の演算は結果がリレーションだから，その結果にさらに演算を適用することができる**

- 例）$\pi_{姓,名}(\sigma_{学科=情報}(R))$　← **内側（選択）を先に**，次に外側（射影）
- 結果（集合表記）：{(赤坂, 泰彦)}

R

射影

選択

| 学籍番号 | 姓 | 名 | 学部 | 学科 | 入学年度 |
|---|---|---|---|---|---|
| 00001 | 秋吉 | 久美子 | 情報 | メディア | 2008 |
| 00002 | 赤坂 | 泰彦 | 情報 | 情報 | 2008 |
| 00003 | 飯島 | 愛 | 情報 | メディア | 2009 |

# 演算の結果を集合表記で書く（型Fの準備）

従業員

| 従業員番号 | 部門番号 | 氏名 | 住所 | 職級 |
|---|---|---|---|---|
| 1 | 1 | 佐藤一郎 | 横浜市 | 2 |
| 2 | 1 | 田中次郎 | 静岡市 | 1 |
| 3 | 2 | 鈴木三郎 | 横浜市 | 2 |
| 4 | 2 | 山田四郎 | 東京都 | 1 |

- $\pi_{\text{部門番号}}(\text{従業員}) = \{\,(1), (2)\,\}$　← 抜き出すと (1), (1), (2), (2)．**重複は1つにまとめる**
- $\sigma_{\text{職級}=1}(\text{従業員}) = \{\,(2, 1, \text{田中次郎}, \text{静岡市}, 1), (4, 2, \text{山田四郎}, \text{東京都}, 1)\,\}$　← 属性はすべて残す

Topic 2

# SQLについて：背景・歴史・利用法など

## トピックのキーワード

- 非手続き型（宣言的）・標準化と方言
- API・コマンドラインツール・埋め込みSQL
- DDL・DML・DCL

# SQLが生まれた背景

- SQL以前は，データベースへの読み書きの手続きをプログラムとして記述していた
  - プログラムとデータ操作が分離していない
  - 例：COBOL言語＋ネットワーク型データベース（CODASYL型）
- データベースの内部構造の変更があると大変
  - データ量や特性が時間と共に変わると内部構造の変更ありえる
  - 最適な読み書き手順を変更する必要が生じる
- 非手続き的なデータベース操作言語が必要となった
  - 欲しいデータを記述し，手続きを書かない（**宣言的**）
  - QUELなどの実験的な言語も提案された
  - 結局IBMが提案したSQLが残った

# SQLの歴史

## 歴史が長く，標準化された技術である

| 時代 | 主な出来事・特徴 |
| --- | --- |
| **1970年代** | IBM研究所の **System R** プロジェクトで **SEQUEL言語** を提案．**1978年** にバージョンアップして **SQL言語** に発展 |
| **1980年代** | 多くのデータベースシステムでSQLが採用され始める．**ISO**, **JIS** による標準化（**SQL87**, **SQL89**） |
| **1990年代** | マルチメディア対応やサーバプログラミング言語との連携等に拡張．**ISO**, **JIS** による標準化（**SQL92**, **SQL99**） |
| **2000年代以降** | **XML拡張**，JSON対応など，さらなる標準化の努力が継続 |

# SQLの特徴

- 機能が豊富な大規模な言語．SQLだけでかなりのことができる
  - データサイエンス・AIで使うデータの前処理にも使われる
- 標準化されているが，すべての標準を実装しているDBMSはない
  - 標準以外の独自の機能はほとんど全部のDBMSが実装している → **方言**に注意
- **非手続き型**である
  - データの読み書き手順をいちいち記述しない．ほしい結果だけを指定する → あとは言語処理系が面倒を見てくれる
- SQLで書かれた「実行可能な」一つの命令を「**文（statement）**」と呼ぶ
  - SELECT文，INSERT文など
- SQLのキーワードは大文字でも小文字でもOK（本講義ではキーワードを大文字で表示する）

# SQLの利用

SQLをプログラミングの中で利用するときには，プログラムとDBMSの連携が必要である．主に，以下の3つの方法がある

1. **APIからの利用**：言語ごとのライブラリを通じてDBMSを呼ぶ．現代では最も標準的
   - 例）Python で SQLite：標準ライブラリ `sqlite3`．PostgreSQL：`psycopg2`．Java：JDBC（2年のデータベース演習で学ぶ）
2. **コマンドラインツールの利用**：DBMS付属のツールにSQLを投入する
   - PostgreSQL の `psql`．pgAdmin のクエリツールもこの仲間（今回使う）
3. **埋め込みSQL**：C言語などのプログラムにSQLを埋め込み，プリプロセッサで変換する
   - 現代ではあまり使われないが，レガシーなプログラムには大量にある

# PythonからのSQLの利用例（APIからの利用）

```
import psycopg2   # PostgreSQLにアクセスするライブラリ

# 接続するための情報（SQL 実習環境ガイドの設定と同じ）
connection = psycopg2.connect(
    host='localhost', database='postgres', user='postgres', password='PASSWORD',
)
cursor = connection.cursor()

# SQL実行: 従業員テーブルのすべての情報を出力する
cursor.execute("SELECT * FROM 従業員;")

for row in cursor:
    print(row)      # (1, 1, '佐藤一郎', '横浜市', 2) 等と表示される

cursor.close()
connection.close()
```

# SQLの分類

- **DDL**（Data Definition Language：データ定義言語）
  - テーブルの作成（CREATE TABLE）・変更（ALTER TABLE）・消去（DROP TABLE）
  - インデックスの作成・変更・消去，ビューの作成・変更・消去，等
- **DML**（Data Manipulation Language：データ操作言語）
  - タプルの追加（INSERT）・変更（UPDATE）・消去（DELETE）
  - タプルの検索（**SELECT**）← 今回・次回の中心
- **DCL**（Data Control Language：データ制御言語）
  - トランザクションの完了・破棄の指示
  - テーブルのロック，権限の付与等

# 注意：リレーションとテーブルの用語の使い分け

- リレーションとテーブルは，本講義では以下の使い分けをする
  - リレーショナルデータモデルにおける表 → **リレーション**
  - SQLにおける表（PostgreSQL上の表） → **テーブル**
- 同様に，タプル → 行（レコード），属性 → 列（カラム）と呼ぶことが多い

Topic 3

# SQLの基本：SELECT文による検索

## トピックのキーワード

- SELECT句・FROM句・WHERE句
- 条件式・文字列の引用符

# SQLの基本：SELECT文

- SELECT文：タプルを検索するためのSQL文（DML：データ操作言語に属する）
- 主に以下の機能を持つが，できることはとても多い
  - テーブルを指定して，そのテーブルに含まれるタプルを出力する
  - 条件を満たすタプルを出力する
  - 属性を選択してタプルを出力する
  - タプルを分類して集計する（次回）
- SELECT文は，「問合せ（query：クエリ）」とも呼ばれる

# SELECT文の基本形

```
SELECT 読む属性 FROM 読むテーブル WHERE どのタプルを読むかの条件 ;
```

- それぞれの「句」を，SELECT句，FROM句，WHERE句と呼ぶ
- 上のSQL文の意味は，
  - FROM句に書かれたテーブルから
  - WHERE句の条件を満たすタプルの
  - SELECT句に書かれた属性を，検索する
- 最後の「;（セミコロン）」は，一つのSQL文の終わりを表す
  - 複数のSQL文を一度に実行するときに，;で区切って複数書く
  - 演習などで一つのSQLを書くときに;を省略しても減点はしない
- 途中に改行を入れてもかまわない．大文字・小文字は区別しない

# SELECT文の例：条件なし

```
SELECT 氏名, 住所 FROM 従業員;
```

- テーブル 従業員 から，（条件なしで）すべてのタプルの，氏名と住所を検索する
- SELECT句には，属性を複数指定できる（カンマで区切る）
- WHERE句は省略できるが，SELECT句とFROM句は省略できない

| 氏名 | 住所 |
|---|---|
| 佐藤一郎 | 横浜市 |
| 田中次郎 | 静岡市 |
| 鈴木三郎 | 横浜市 |
| 山田四郎 | 東京都 |

# SELECT文の例：すべての属性を検索する

## 「*（アスタリスク）」は，すべての属性を表す

```
SELECT * FROM 従業員;
```

- テーブル 従業員 から，（条件なしで）すべてのタプルの，すべての属性を検索する

| 従業員番号 | 部門番号 | 氏名 | 住所 | 職級 |
|---|---|---|---|---|
| 1 | 1 | 佐藤一郎 | 横浜市 | 2 |
| 2 | 1 | 田中次郎 | 静岡市 | 1 |
| 3 | 2 | 鈴木三郎 | 横浜市 | 2 |
| 4 | 2 | 山田四郎 | 東京都 | 1 |

# SELECT文の例：条件指定

```
SELECT * FROM 従業員 WHERE 住所 = '横浜市';
```

- テーブル 従業員 から，属性 住所 の値が「横浜市」であるタプルの，すべての属性を検索する
- WHERE句の中は，条件式を入れる．条件式は真偽値を返却する式
  - 文字列は **シングルクォート** `'…'` で囲む．数値は囲まない
  - `=`，`<>`（等しくない），`<`，`<=`，`>`，`>=` が使える．例：`職級 >= 2`

| 従業員番号 | 部門番号 | 氏名 | 住所 | 職級 |
|---|---|---|---|---|
| 1 | 1 | 佐藤一郎 | 横浜市 | 2 |
| 3 | 2 | 鈴木三郎 | 横浜市 | 2 |

# SELECT文の例：条件指定＋属性の指定

```
SELECT 氏名 FROM 従業員 WHERE 職級 = 1;
```

- テーブル 従業員 から，職級が 1 であるタプルの，氏名を検索する
- リレーショナル代数で書くと $\pi_{\text{氏名}}(\sigma_{\text{職級}=1}(\text{従業員}))$
  - WHERE句が選択，SELECT句が射影に対応する（対応関係は次回くわしく）

| 氏名 |
| --- |
| 田中次郎 |
| 山田四郎 |

# やってみよう：サンプルデータの投入（10分）

学生ポータルの第4回ページ「配布資料」から2つのファイルをダウンロードし，pgAdmin のクエリツールで開いて実行する（ガイドの 5.）

1. `sample.sql`（日本語の5表：部門・従業員・部品・業者・供給）→ 開いて F5
2. `chinook_pg.sql`（音楽販売 Chinook の11表）→ 開いて F5（数秒かかる）
3. 確認：次を実行して，4行の表が出れば OK

```
SELECT * FROM 従業員;
```

- どちらも何度実行してもよい（表を作り直してから入れ直す）
- Colab の人：ノートブックを上から実行すれば同じデータができる
- 終わったら，演習2 の SQL をその場で試してよい

# Chinook（音楽販売）のテーブルでも試す

```
SELECT first_name, last_name, title FROM employee WHERE city = 'Calgary';
```

- 英語のデータ．表名・列名は小文字．artist・album・track（曲，約3500件）・customer・invoice など
- 次回，複数のテーブルの結合や集計はこのデータでも練習する

| first_name | last_name | title |
|---|---|---|
| Nancy | Edwards | Sales Manager |
| Jane | Peacock | Sales Support Agent |
| …（全5行） | | |

Topic 4

# 今回の演習と小テストの「型」

# 今回の小テストの「型」（小テスト2：11/25）

- **型F：リレーショナル代数の結果**　演算の結果を集合表記 { (値, …), … } で書く
  - 例）$\pi_{\text{部門番号}}(\text{従業員}) = \{ (1), (2) \}$　← 重複は1つにまとめる
  - 例）$\pi_{\text{氏名}}(\sigma_{\text{部門番号}=2}(\text{従業員})) = \{ (\text{鈴木三郎}), (\text{山田四郎}) \}$　← 内側から順に
  - 手順：射影＝列を抜く → 重複をまとめる．選択＝行を選ぶ → 列はそのまま．結合＝条件が成り立つ組だけ
- **型G：SELECT文（基本）**　日本語の問合せを SELECT・FROM・WHERE で書く
  - 例）住所が横浜市である従業員の氏名と職級
    → `SELECT 氏名, 職級 FROM 従業員 WHERE 住所 = '横浜市';`
  - 手順：「〜の」が属性 → SELECT句．表の名前 → FROM句．「〜である」が条件 → WHERE句

# 第4回演習と次回

- ポイント資料の裏面を，表面の例にならって自分の手で解く
  - 演習1：型Fの類題（6題）　演習2：型Gの類題（4題．**実際に実行して行数を書く**）　確認問題2題（環境の確認など）
- 提出：表面・裏面を撮影して WebClass「第4回 演習」へ．締切：**次回（10/28）の講義開始30分後**
- **小テスト1：次回（10/28）の講義の最後15分**
  - 範囲：第1〜3回．持ち込みは第1〜3回のポイント資料（3枚）のみ．解答は別紙の解答用紙に書く
- 環境が未解決の人：次回までに講義の前後（または研究室）で教員に相談する．それまでは Colab で演習を進める
- 次回の予習：教科書 第5章 p.48〜62（SQL）